

EXPERIMENT 3: SURVEY REPORT ON EMERGING TRENDS IN LOW-CODE APPLICATION DEVELOPMENT

Laboratory Report • Department of Computer Science & Engineering

STUDENT NAME	REGISTER NO.	DEPARTMENT	COURSE
Gayathri Boopathy	240701141	CSE	Low Code Application Development
EXPERIMENT	EXP 3		

1 AIM

To conduct a comprehensive survey on emerging trends in low-code application development, analyze the operational shift from traditional software development methodologies to low-code/no-code platforms, evaluate key benefits and organizational challenges, compare industry-standard platforms, and demonstrate practical application creation through a customized Vehicle Service Request application interface.

This report surveys the transition from hand-coded development toward visual, component-based and automated low-code application development.

TRADITIONAL APPLICATION DEVELOPMENT VS EMERGING LOW-CODE DEVELOPMENT

Traditional software engineering relies heavily on hand-coded programming, complex build tools, continuous integration pipelines, and specialized developer teams. Emerging Low-Code Application Development Platforms (LCADPs) transform this paradigm through drag-and-drop visual interfaces, pre-built components, and automated deployment mechanisms.

Parameter	Traditional Development	Low-Code Development
Development Speed	Slow; months to years due to extensive manual coding and testing.	Rapid; hours to weeks using drag-and-drop elements and template workflows.
Skill Requirements	High; requires proficient software engineers, DevOps, and DBAs.	Accessible; enables "Citizen Developers" and business analysts alongside engineers.
Cost & Maintenance	High initial investment and long-term technical-debt maintenance cost.	Reduced upfront cost; automated platform updates lower maintenance burden.
Integration & Flexibility	Fully customizable; custom API connectors and microservices support.	Modular flexibility; pre-built connectors with room for custom scripting.

BENEFITS AND CHALLENGES IN LOW CODE

✓ KEY BENEFITS

- **Accelerated Time-to-Market:** Rapid prototype-to-production cycles.
- **Democratization of IT:** Bridges the gap between business needs and developer resources.
- **Cost Efficiency:** Minimizes development backlog and specialized hiring needs.
- **Standardized Governance:** Centralized control and built-in security compliance.

■ KEY CHALLENGES

- **Vendor Lock-in:** Migration difficulty between platform proprietary formats.
- **Customization Limits:** Complex edge cases may require platform extensions.
- **Shadow IT Risks:** Governance issues if unmanaged applications proliferate.
- **Security & Compliance:** Reliance on platform-level security guarantees.

4

COMPARISON OF EXISTING LOW-CODE PLATFORMS

A comparative evaluation of leading low-code application development platforms, highlighting target user bases, scripting capability, primary deployment channels, and key enterprise strengths.

Platform	Target Audience	Scripting Engine	Deployment	Key Strength
Zoho Creator	SMBs & Enterprise teams	Deluge Scripting	Web, Mobile (iOS/Android)	Rapid form-centric database apps
Microsoft Power Apps	Enterprise & M365 users	Power Fx / C#	Web, Teams, Mobile	Deep ecosystem & Azure integration
OutSystems	Professional Developers	JavaScript / C#	Cloud, On-Premise, Mobile	High scalability & enterprise tech debt care
Mendix	Enterprise Cross-teams	Java / JavaScript	Multi-Cloud, Hybrid	Robust visual domain modeling
Appian	Enterprise BPM teams	SAIL Expression Language	Cloud, On-Premise	Advanced Process Automation & AI

5

PRACTICAL IMPLEMENTATION: VEHICLE SERVICE REQUEST APP

The practical component demonstrates the design of a custom Vehicle Service Request Form using a low-code platform environment, establishing automated database schema creation, input validation rules, and an intuitive interface layout.

Figure 3.1: Vehicle Service Request Form Interface (Owner, Contact & Registration Details)

The screenshot displays a mobile application interface titled "My first Creator app" with a sidebar menu containing "Home", "All Requests", "Add Request", "Vehicle Service Requests", "Vehicle Service Request Form", and "All Vehicle Service Requests". The main content area is titled "Vehicle Service Request Form" and contains the following fields:

- Issue_Description**: A large text input field.
- Odometer_Reading**: A text input field with a placeholder "#####".
- Preferred_Service_Date**: A date input field with a placeholder "dd-MMM-yyyy".
- Preferred_Dropoff_Time**: A time input field with a placeholder "HH:mm:ss".
- Vehicle_Type**: A dropdown menu with the option "-Select-".
- Service_Type**: A group of three radio buttons labeled "Routine Service", "Breakdown Repair", and "Accident Repair".
- Additional_Services_Requested**: A dropdown menu with the option "-Select-".
- Authorize_Estimate_And_Repair**: A checkbox labeled "Agreed".

A blue "Submit" button is located at the bottom center of the form.

Figure 3.2: Vehicle Service Request Form Interface (Service Selection, Booking Time & Authorization)

Outcome

The survey report on emerging trends in low-code application development was successfully completed. The analysis highlighted the operational shift from traditional development to low-code solutions, along with their associated benefits, challenges, and platform comparisons. Furthermore, a Vehicle Service Request Form application was designed and configured using low-code form builders and data binding mechanisms.

Conclusion: Low-code application development provides a visual and modular approach that can shorten development cycles and broaden participation in application creation, while still requiring attention to governance, customization, vendor dependence, security, and compliance.

Student Signature

Faculty Signature

Gayathri Boopathy